

An Analysis of Recurrent neural network

Recurrent neural network

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

Recurrent neural network -- Part 1

$$\tau_{i,y} = -y_i + \sum_{j=1}^n w_{ji} \sigma(y_j - \Theta_j) + I_i(t)$$
$$\tau_{i,y} = -y_i + \sum_{j=1}^n w_{ji} \sigma(y_j - \Theta_j) + I_i(t)$$

Recurrent neural network -- Part 2

$$h_t = \sigma(W_h x_t + U_h h_{t-1} + b_h) \quad y_t = \sigma_y(W_y h_t + b_y)$$
$$h_t = \sigma(W_h x_t + U_h h_{t-1} + b_h) \quad y_t = \sigma_y(W_y h_t + b_y)$$

Recurrent neural network -- Part 3

A bidirectional RNN (biRNN) is composed of two RNNs, one processing the input sequence in one direction, and another in the opposite direction. Abstractly, it is structured as follows: The forward RNN processes in one direction: $f_\theta(x_0, h_0) = (y_0, h_1)$, $f_\theta(x_1, h_1) = (y_1, h_2)$, ... $f_\theta(x_{n-1}, h_{n-1}) = (y_{n-1}, h_n)$. The backward RNN processes in the opposite direction: $f_{\bar{\theta}}(x_{n-1}, h_{n-1}) = (\bar{y}_{n-1}, \bar{h}_{n-2})$, $f_{\bar{\theta}}(x_{n-2}, \bar{h}_{n-2}) = (\bar{y}_{n-2}, \bar{h}_{n-3})$, ... $f_{\bar{\theta}}(x_0, \bar{h}_0) = (\bar{y}_0, \bar{h}_1)$.

Recurrent neural network -- Part 4

Machine translation Robot control Time series prediction Speech recognition Speech synthesis Brain-computer interfaces Time series anomaly detection Text-to-Video model Energy forecasting Rhythm learning Music composition Grammar learning Handwriting recognition Human action recognition Protein homology detection Predicting subcellular localization of proteins Several prediction tasks in the area of business process management Prediction in medical care pathways Predictions of fusion plasma disruptions in reactors (Fusion Recurrent Neural Network (FRNN) code)

Recurrent neural network -- Part 5

Continuous-time A continuous-time recurrent neural network (CTRNN) uses a system of ordinary differential equations to model the effects on a neuron of the incoming inputs. They are typically analyzed by dynamical systems theory. Many RNN models in neuroscience are continuous-time. For a neuron i in the network with activation y_i , the rate of change of activation is given by:

Recurrent neural network -- Part 6

Recurrent multilayer perceptron network Generally, a recurrent multilayer perceptron network (RMLPN) consists of cascaded subnetworks, each containing multiple layers of nodes. Each subnetwork is feed-forward except for the last layer, which can have feedback connections. Each of these subnets is connected only by feed-forward connections.

Recurrent neural network -- Part 7

When the neural network has learned a certain percentage of the training data. When the minimum value of the mean-squared-error is satisfied. When the maximum number of training generations has been reached. The fitness function evaluates the stopping criterion as it receives the mean-squared error reciprocal from each network during training. Therefore, the goal of the genetic algorithm is to maximize the fitness function, reducing the mean-squared error. Other global (and/or evolutionary) optimization techniques may be used to seek a good set of weights, such as simulated annealing or particle swarm optimization.

Recurrent neural network -- Part 8

Apache Singa Caffe: Created by the Berkeley Vision and Learning Center (BVLC). It supports both CPU and GPU. Developed in C++, and has Python and MATLAB wrappers. Chainer: Fully in Python, production support for CPU, GPU, distributed training. Deeplearning4j: Deep learning in Java and Scala on multi-GPU-enabled Spark. Flux: includes interfaces for RNNs, including GRUs and LSTMs, written in Julia. Keras: High-level API, providing a wrapper to many other deep learning libraries. Microsoft Cognitive Toolkit MXNet: an open-source deep learning framework used to train and deploy deep neural networks. PyTorch: Tensors and Dynamic neural networks in Python with GPU acceleration. TensorFlow: Apache 2.0-licensed Theano-like library with support for CPU, GPU and Google's proprietary TPU, mobile Theano: A deep-learning library for Python with an API largely compatible with the NumPy library. Torch: A scientific computing framework with support for machine learning algorithms, written in C and Lua.

Recurrent neural network -- Part 9

Long short-term memory (LSTM) is the most widely used RNN architecture. It was designed to solve the vanishing gradient problem. LSTM is normally augmented by recurrent gates called "forget gates". LSTM prevents backpropagated errors from vanishing or exploding. Instead, errors can flow backward through unlimited numbers of virtual layers unfolded in space. That is, LSTM can learn tasks that require memories of events that happened thousands or even millions of discrete time steps

An Analysis of Recurrent neural network

Recurrent neural network

earlier. Problem-specific LSTM-like topologies can be evolved. LSTM works even given long delays between significant events and can handle signals that mix low and high-frequency components. Many applications use stacks of LSTMs, for which it is called "deep LSTM". LSTM can learn to recognize context-sensitive languages unlike previous models based on hidden Markov models (HMM) and similar concepts.

Recurrent neural network -- Part 10

Teacher forcing An RNN can be trained into a conditionally generative model of sequences, aka autoregression. Concretely, let us consider the problem of machine translation, that is, given a sequence $(x_1, x_2, \dots, x_n)$ of English words, the model is to produce a sequence $(y_1, \dots, y_m)$ of French words. It is to be solved by a seq2seq model. Now, during training, the encoder half of the model would first ingest $(x_1, x_2, \dots, x_n)$, then the decoder half would start generating a sequence $(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_l)$. The problem is that if the model makes a mistake early on, say at $\hat{y}_2$, then subsequent tokens are likely to also be mistakes. This makes it inefficient for the model to obtain a learning signal, since the model would mostly learn to shift $\hat{y}_2$ towards y_2 , but not the others. Teacher forcing makes it so that the decoder uses the correct output sequence for generating the next entry in the sequence. So for example, it would see $(y_1, \dots, y_k)$ in order to generate $\hat{y}_{k+1}$.

Recurrent neural network -- Part 11

A recursive neural network is created by applying the same set of weights recursively over a differentiable graph-like structure by traversing the structure in topological order. Such networks are typically also trained by the reverse mode of automatic differentiation. They can process distributed representations of structure, such as logical terms. A special case of recursive neural networks is the RNN whose structure corresponds to a linear chain. Recursive neural networks have been applied to natural language processing. The recursive neural tensor network uses a tensor-based composition function for all nodes in the tree.